

UserControls

UserControls vs. CustomControls

- Beides Mechaniken zur Erstellung von eigenen Controls
 - unterschiedliche Komplexität -> unterschiedliche ‚Fähigkeiten‘
- UserControl wenn das Control...
 - ... nur aus bereits bestehenden Controls aufgebaut sein soll
 - ... keine komplexen Anpassungsfähigkeiten benötigt
 - ... wenige/keine einzigartige Events benötigt
 - ... keine unterschiedlichen visuellen Zustände braucht
- CustomControl wenn das Control...
 - ... Anpassbarkeit mittels Templates benötigt
 - ... Themes unterstützen soll
 - ... viele einzigartige Events benötigt
 - ... unterschiedliche visuelle Zustände beinhalten soll

UserControls und CustomControls sind zwei Arten von benutzerdefinierten Steuerelementen in WPF, die verschiedene Ansätze zur Erstellung und Verwendung bieten.

Ein UserControl ist ein benutzerdefiniertes Steuerelement, das aus einer Kombination von XAML-Code und Code-Behind besteht. Es ermöglicht die Zusammenstellung verschiedener vorhandener Steuerelemente und Layouts zu einer wiederverwendbaren Einheit. Ein UserControl bietet eine bequeme Möglichkeit, eine bestimmte Benutzeroberfläche oder Funktion zu definieren und an verschiedenen Stellen in einer Anwendung zu verwenden. Es ermöglicht die Kapselung von XAML und Code in einer einzelnen Einheit und erleichtert so die Wartung und Wiederverwendung.

Ein CustomControl hingegen ist ein benutzerdefiniertes Steuerelement, das auf einer vorhandenen Basisklasse wie Control oder FrameworkElement basiert. Es ermöglicht die Erstellung eines völlig neuen Steuerelements mit eigener Darstellung und Verhalten. Im Gegensatz zum UserControl erfordert die Erstellung eines CustomControls oft eine umfangreichere Implementierung, einschließlich der Definition von visuellen Zuständen, der Behandlung von Ereignissen und der Verwaltung von Abhängigkeitseigenschaften. CustomControls bieten eine höhere Flexibilität und Kontrolle über das Aussehen und Verhalten des Steuerelements, sind jedoch in der Regel komplexer zu erstellen und zu warten.

In den meisten Fällen genügt die Erstellung eines UserControls.

UserControl

- Erbt über ContentControl von Control
- Aussehen lässt sich nicht über Template anpassen
- .xaml + .xaml.cs Datei (Wie MainWindow)
- Aussehen fest mit Logik verdrahtet
- Abgeschlossenes Control ist eine Einheit
 - > d.h. auf einzelne Teile kann nicht mehr zugegriffen werden
- DependencyProperties um Bindings zu ermöglichen

Die UserControl-Klasse ist eine Basisklasse, mit der benutzerdefinierte Steuerelemente erstellt werden können. Ein UserControl besteht aus einer Kombination von XAML-Code und Code-Behind, die zusammen eine wiederverwendbare Benutzeroberfläche definieren.

Die UserControl-Klasse erbt von der ContentControl-Klasse und bietet zusätzliche Funktionen, um die Erstellung und Verwendung benutzerdefinierter Steuerelemente zu erleichtern. Ein UserControl kann in XAML definiert werden, indem verschiedene vorhandene Steuerelemente und Layouts verwendet werden, um die gewünschte Benutzeroberfläche zu gestalten. Codeseitig können ihm eigene DependencyProperties, Events und Funktionen hinzugefügt werden.

Die Anwendung eines UserControls besteht darin, es in anderen XAML-Dateien einzubetten oder programmatisch zu instanziiieren. Ein UserControl kann als eigenständiges Steuerelement verwendet werden, das spezifische Funktionen oder Darstellungen bereitstellt. Es kann auch als Container dienen, um andere Steuerelemente zu gruppieren und eine zusammengehörige Benutzeroberfläche zu erstellen.

Ein Vorteil der Verwendung von UserControls liegt in ihrer Wiederverwendbarkeit. Sobald ein UserControl erstellt wurde, kann es in verschiedenen Teilen der Anwendung verwendet werden, um konsistente Benutzeroberflächen zu gewährleisten und den Entwicklungsprozess zu beschleunigen.

Dependency Property

- Globales Dictionary bestehend aus Schlüssel - Wert - Paaren
- WPF-Controls verwendet statt normalen Properties eine spezielle Variante
 - Integrierter Benachrichtigungsmechanismus („Change Notification“) -> Quelle für Binding
 - Besitzen Default-Wert
- Sind dynamischer als .NET Properties
- Werden benötigt für:
 - Bindings
 - Styles
 - Trigger
 - Animationen

DependencyProperties sind ein zentrales Konzept in WPF, das die effiziente Verwaltung von Eigenschaften in benutzerdefinierten Steuerelementen ermöglicht. Sie erweitern die Funktionalität herkömmlicher .NET-Eigenschaften, indem sie die Unterstützung für Funktionen wie Datenbindung, Animationen, Styles und Templates bieten.

Der Hauptzweck von DependencyProperties besteht darin, die effiziente Aktualisierung und Verwaltung von Eigenschaftswerten zu ermöglichen. Im Gegensatz zu herkömmlichen .NET-Eigenschaften werden die Werte von DependencyProperties nicht direkt in den Instanzen der Steuerelemente gespeichert, sondern in einem zentralen Speicher auf dem Objekt namens "DependencyObject". Dies ermöglicht es, dass Eigenschaftswerte automatisch aktualisiert und zwischen verschiedenen Instanzen eines Steuerelements gemeinsam genutzt werden können.

DependencyProperties unterstützen auch das Konzept des Wertesystems, das es ermöglicht, Werte aus verschiedenen Quellen zu beziehen und diese zu priorisieren. Ein Wert kann beispielsweise durch direkte Festlegung, Datenbindung, Stil oder Vorlage gesetzt werden. Das Wertesystem von DependencyProperties löst automatisch Konflikte zwischen verschiedenen Quellen auf und stellt sicher, dass der richtige Wert verwendet wird.

Ein weiterer wichtiger Aspekt ist ihre Fähigkeit, Änderungen an ihren Werten zu überwachen und darauf zu reagieren. Dies ermöglicht die Implementierung von Rückrufmethoden, die bei Wertänderungen ausgelöst werden können, und gewährleistet die automatische Aktualisierung von Bindungen.

DependencyProperty - Implementierung

```
// Kapselung DP für Zugriff in xaml
public Uri Source
{
    get { return (Uri)GetValue(SourceProperty); }
    set { SetValue(SourceProperty, value); }
}

// Registrierung DependencyProperty
public static readonly DependencyProperty SourceProperty;
SourceProperty = DependencyProperty.Register("Source", typeof(Uri),
    typeof(VideoPlayer), new FrameworkPropertyMetadata(null, new
    PropertyChangedCallback(OnSourceChanged)));
```

Der obige Codeausschnitt zeigt die Implementierung einer Dependency Property (DP) mit dem Namen "Source" in einer benutzerdefinierten Klasse namens "VideoPlayer". Um den Zugriff auf die DP in XAML zu ermöglichen, wird eine herkömmliche .NET-Eigenschaft "Source" definiert, die den Getter und Setter verwendet, um den Wert der DP abzurufen und zu setzen. Der Getter ruft den Wert der DP über die Methode "GetValue" ab, während der Setter den Wert über die Methode "SetValue" setzt. Beides sind Methoden der ererbten Klasse 'DependencyObject', welche eine Voraussetzung für die Implementierung von DependencyProperties ist.

Die Registrierung der DependencyProperty erfolgt über die statische Methode "DependencyProperty.Register". Dabei wird der Name der DP ("Source"), ihr Typ (Uri), der Typ der Klasse, die die DP verwendet (VideoPlayer), und Metadaten angegeben. Die Metadaten werden durch eine Instanz der Klasse "FrameworkPropertyMetadata" definiert. Hier wird der anfängliche Wert der DP auf "null" gesetzt und eine Rückrufmethode ("OnSourceChanged") angegeben, die aufgerufen wird, wenn sich der Wert der DP ändert.

Durch die Implementierung einer DP in dieser Weise können Entwickler den Wert der DP in XAML festlegen und auf Änderungen reagieren, indem sie die entsprechende Syntax verwenden, z. B. "<VideoPlayer Source='...'>". Gleichzeitig wird die Interoperabilität mit anderen Funktionen wie Datenbindung und Vorlagen in WPF gewährleistet, da DependencyProperties die integrierten Mechanismen von WPF nutzen.

Content - Property

- UserControl erbt automatisch von ContentControl
 - -> jedes UserControl besitzt automatisch die Content-Eigenschaft
- Content überschreibt standardmäßig den VisualTree
- Um dies zu verhindern muss die Content-Eigenschaft umgeleitet werden

```
[ContentProperty("MyContent")]
public partial class ColorPicker : UserControl
{
    ...
    public object MyContent
    {
        get { return (object)GetValue(MyContentProperty); }
        set { SetValue(MyContentProperty, value); }
    }

    public static readonly DependencyProperty MyContentProperty =
        DependencyProperty.Register(...);
}
```

Da UserControls automatisch von der ContentControl-Klasse erben, besitzen diese immer auch die Content-Eigenschaft. Diese ist auch die Default-Eigenschaft, welche besetzt wird, wenn etwas direkt zwischen Start- und Endtag des UserControls gelegt wird. Leider wird hier standardmäßig der komplette VisualTree des UserControls überschrieben.

Um dies zu verhindern muss die Default-Property umgeleitet werden. Dies erfolgt über den Einsatz des Attributs ‚ContentProperty‘, in welchem jede beliebige DependencyProperty des UserControls angegeben werden kann. Diese kann dann innerhalb des XAML-Markups an gewünschter Stelle angebunden werden.

Routed Event - Implementierung

- Eigene Events müssen unter Angabe des EventHandlers implementiert werden
- Registrierung über EventManager unter Angabe der Routing-Strategie (Bubble, Tunnel, Direct)
- Auslöser muss individuell implementiert werden (z.B. als PropertyChanged-Methode in DPs)

```
public event RoutedPropertyChangedEventHandler<Uri> SourceChanged
{
    add { AddHandler(SourceChangedEvent, value); }
    remove { RemoveHandler(SourceChangedEvent, value); }
}

public static readonly RoutedEvent SourceChangedEvent =
    EventManager.RegisterRoutedEvent( "PickedColorChanged",
    RoutingStrategy.Direct, typeof(RoutedPropertyChangedEventHandler<Uri>),
    typeof(VideoPlayer));
```

Das obige Codebeispiel definiert ein Ereignis mit dem Namen "SourceChanged" vom Typ "RoutedPropertyChangedEventHandler<Uri>". Das Ereignis soll ausgelöst werden, wenn sich der Wert der "Source"-Eigenschaft ändert. Dieser Auslöser muss separat programmiert werden.

Die Event-Accessoren "add" und "remove" werden verwendet, um Ereignisbehandler zum Hinzufügen und Entfernen von Handlern zu verwalten. Die aufgerufenen Methoden sind in der ererbten Klasse `UIElement` definiert.

Die "SourceChangedEvent"-Eigenschaft ist vom Typ "RoutedEvent" und wird verwendet, um das Ereignis zu registrieren. Die Methode "RegisterRoutedEvent" des "EventManager" wird verwendet, um das Ereignis mit dem Namen "SourceChanged" zu registrieren. Das RoutingStrategy-Argument legt die Art des Ereignisroutings fest, in diesem Fall "Direct". Der Typ des Ereignisargs wird als "RoutedPropertyChangedEventHandler<Uri>" angegeben. Das vierte Argument gibt den Typ des Elements an, auf dem das Ereignis ausgelöst wird, in diesem Fall „VideoPlayer“.

Im XAML-Markup kann an diesem Event nun eine Handler-Methode angemeldet werden, welche beim Auslösen ausgeführt wird. Dieses Auslösen muss an geeigneter Stelle definiert werden. Beispielsweise kann dies in der entsprechenden DependencyProperty angegeben werden oder es kann z.B. ein anderes Event eines Teil-Controls des UserControls verwendet werden.

Attached Property - Implementierung

```
public static int GetCount(DependencyObject obj)
{
    return (int)obj.GetValue(CountProperty);
}

public static void SetCount(DependencyObject obj, int value)
{
    obj.SetValue(CountProperty, value);
}

public static readonly DependencyProperty CountProperty =
    DependencyProperty.RegisterAttached("Count", typeof(int),
        typeof(VideoPlayer), new PropertyMetadata(0));
```

Das obige Codebeispiel zeigt die Implementierung einer angefügten (attached) Eigenschaft in WPF.

Die Methode "GetCount" ermöglicht den Zugriff auf den Wert der angefügten Eigenschaft "Count" eines bestimmten "DependencyObject"-Objekts. Dabei wird der Wert über die "GetValue" Methode abgerufen und als "int" zurückgegeben. Das übergebene DependencyObject ist das Objekt, für welches der neue Wert gesetzt wird.

Die Methode "SetCount" wird verwendet, um den Wert der angefügten Eigenschaft "Count" für ein bestimmtes "DependencyObject"-Objekt festzulegen. Dabei wird der Wert über die "SetValue" Methode zugewiesen.

Die "CountProperty" ist eine statische "DependencyProperty" und wird verwendet, um die angefügten Eigenschaften zu registrieren. Durch die Verwendung der Methode "RegisterAttached" des "DependencyProperty"-Typs wird die angefügte Eigenschaft mit dem Namen "Count" und dem Typ "int" für das "VideoPlayer"-Element registriert. Die "PropertyMetadata" definiert dabei das Standardverhalten der Eigenschaft, in diesem Fall einen anfänglichen Wert von 0.

Attached Properties ermöglichen es, zusätzliche Eigenschaften zu einem beliebigen WPF-Element hinzuzufügen, ohne die Klasse des Elements selbst zu erweitern. Diese Eigenschaften können dann in XAML oder im Code verwendet werden, um zusätzliche Informationen oder Zustände zu speichern und abzurufen. Das Setzen dieser Attached Property in XAML erfolgt nach dem Muster `,VideoPlayer.Count = " ... "`.